



Laporan Audit Keamanan

Autentikasi API Streaming CCTV

Sistem

CCTV Server + Cloudflare Tunnel

Tanggal Audit

16 Februari 2026

Domain

`api.foodiserver.my.id`

Klasifikasi

INTERNAL — CONFIDENTIAL

⚠ Ringkasan Eksekutif

Bagian 1 — Audit Kondisi Saat Ini: Ditemukan **3 temuan kritis**, **3 sedang**, **2 area baik**.

Bagian 2 — Rencana Per-Camera Token: Arah bagus ✅, tapi ada **3 risiko kritis** yang harus dimitigasi sebelum implementasi.

📄 Bagian 1

Audit Middleware Auth
Kondisi server saat ini

⏳ Bagian 2

Analisis Rencana
Per-Camera Auth Token

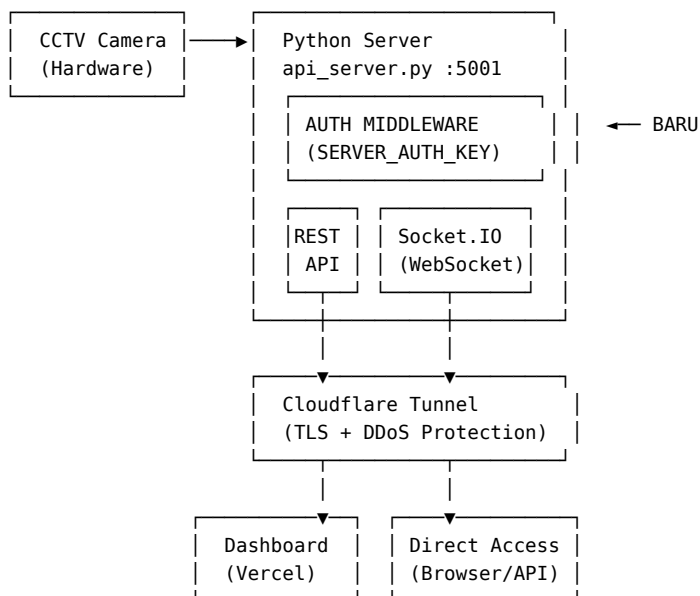
BAGIAN 1

Audit Middleware Authentication — Kondisi Saat Ini

Arsitektur Sistem

1. Diagram Alur Data

Berikut adalah aliran data dari kamera CCTV hingga ke pengguna akhir:



2. Yang Dilindungi & Metode Auth

DILINDUNGI

- 📺 Video Stream (MJPEG)
- 📷 Snapshot
- ⚙️ Pengaturan (FPS, Quality)
- 📊 Data Statistik Kendaraan
- 🛑 Kontrol Start/Stop

TERBUKA (Sengaja)

- ❤️ /api/health — monitoring uptime

METODE AUTH

- x-api-key header **AMAN**
- ?auth= query param **BERISIKO**

Ringkasan Temuan (Bagian 1)

ID	Temuan	Severity	Status
F-01	WebSocket tanpa autentikasi	KRITIS	OPEN
F-02	Dashboard tidak kirim auth header	KRITIS	OPEN
F-03	Credential bocor di URL query param	KRITIS	OPEN
F-04	Tidak ada rate limiting auth	SEDANG	OPEN
F-05	Timing-unsafe comparison	SEDANG	OPEN
F-06	Single static key, tanpa rotasi	SEDANG	OPEN
F-07	Cloudflare Tunnel (TLS/DDoS)	BAIK	OK
F-08	Security headers di Vercel	BAIK	OK

3

KRITIS

3

SEDANG

2

BAIK

Detail Temuan

KRITIS

F-01 — WebSocket (Socket.IO) Tanpa Autentikasi

Lokasi: dashboard/src/hooks/useWebSocket.js baris 33

Masalah: Koneksi Socket.IO tidak mengirimkan credential. Siapapun yang tahu URL bisa konek dan menerima data real-time.

JavaScript — SEKARANG

```
socketRef.current = io(SOCKET_URL, {
  transports: ['websocket', 'polling'],
  // ❌ TIDAK ADA AUTH
});
```



Penjelasan Sederhana

Kamu pasang gembok di pintu depan (REST), tapi jendela belakang (WebSocket) terbuka lebar.

✓ Rekomendasi

JavaScript — Client FIX

```
io(SOCKET_URL, {
  auth: { token: import.meta.env.VITE_SERVER_AUTH_KEY }
});
```

Python — Server FIX

```
@socketio.event
def connect(auth):
    if KEY and auth.get('token') != KEY:
        raise ConnectionRefusedError('Unauthorized')
```

KRITIS

F-02 — Dashboard Tidak Kirim Auth Header ke Server

Lokasi: dashboard/src/services/api.js baris 7-10

Axios instance **tanpa auth header**. Semua API call gagal 401 begitu server auth diaktifkan.

✓ Rekomendasi

JavaScript — FIX

```
const AUTH = import.meta.env.VITE_SERVER_AUTH_KEY || '';
const api = axios.create({
  baseURL: API_BASE_URL,
```

```
headers: AUTH ? { 'x-api-key': AUTH } : {},  
});
```

KRITIS**F-03 — Credential Bocor di URL Query Parameter**

URL ?auth=PASSWORD tersimpan di: browser history, server logs, Cloudflare logs, referer header, bookmark sync, proxy logs.

 Penjelasan Sederhana

Menulis password di papan nama depan rumah — semua yang lewat bisa baca.

 Rekomendasi

- Utamakan x-api-key header
- ?auth= hanya untuk MJPEG stream (tidak bisa kirim header)
- Idealnya: cookie-based auth + /api/login endpoint

SEDANG**F-04 — Tidak Ada Rate Limiting**

Penyerang bisa brute-force tanpa batas. **Fix:** Cloudflare WAF rule + flask-limiter (5 gagal/menit = blokir 15 menit).

SEDANG**F-05 — Timing-Unsafe Comparison**

String == bocorkan info waktu per karakter. **Fix:** hmac.compare_digest(provided, expected).

SEDANG**F-06 — Single Static Key, Tanpa Rotasi**

1 password untuk semua. Bocor 1 = ganti semua. **Fix sekarang:** dokumentasi rotasi. **Fix nanti:** JWT per-user.

Yang Sudah Baik 

F-07: Cloudflare Tunnel

- ✓ TLS (HTTPS) ✓ DDoS protection
- ✓ IP tersembunyi ✓ Tidak exposed langsung

F-08: Vercel Headers

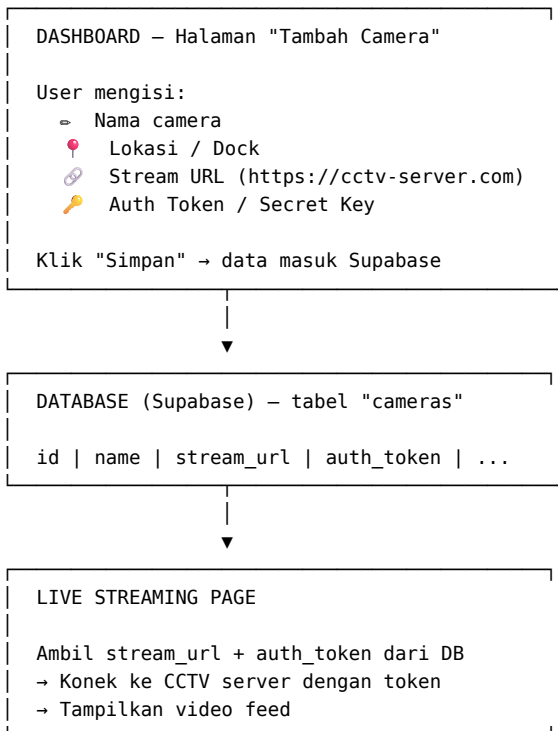
- ✓ nosniff ✓ X-Frame-Options: DENY
- ✓ XSS Protection ✓ Referrer-Policy
- ✗ Belum ada: Content-Security-Policy

BAGIAN 2

Analisis Rencana: Per-Camera Auth Token

Pemahaman Rencana

Alur yang Direncanakan



Penilaian: Apa yang Bagus?

✅ Kelebihan Rencana Per-Camera Token

- 1 Per-camera credential** — Jika 1 token bocor, camera lain tetap aman
- 2 Self-service** — Operator bisa tambah camera tanpa edit .env atau restart server
- 3 Multi-server** — Bisa konek ke banyak server CCTV berbeda, masing-masing punya auth sendiri
- 4 RLS sudah ada** — Tabel cameras sudah punya Row Level Security per tenant di Supabase
- 5 Jauh lebih baik** dari single static key (F-06) — ini evolusi yang benar

Risiko yang Harus Dimitigasi

KRITIS

R-01 — Token Tersimpan Plaintext di Database

Masalah: Jika `auth_token` disimpan sebagai kolom TEXT biasa di tabel `cameras`:

- 🔓 Siapapun dengan akses Supabase Dashboard bisa lihat semua token
- 💧 SQL injection / data breach = semua CCTV terbuka
- 🔑 Supabase anon key + RLS bypass = bencana

- 👁️ Token muncul di database backup
- 📄 Token muncul di SELECT query biasa
- 🔍 Token tercatat di query logs

💡 Penjelasan Sederhana

Bayangkan kamu simpan semua kunci rumah pelanggan di laci yang tidak dikunci. Siapa saja yang buka laci itu, langsung bisa masuk ke semua rumah.

✅ Rekomendasi

- **Minimum:** Enkripsi token sebelum simpan. Decrypt hanya saat dibutuhkan di server-side
- **Lebih baik:** Gunakan Supabase Vault untuk menyimpan secrets
- **JANGAN** return kolom `auth_token` ke client via query biasa

KRITIS

R-02 — Token Terekspos ke Browser Client

Masalah: Dengan arsitektur saat ini, aliran data akan seperti ini:

❌ ARSITEKTUR BERBAHAYA:

Browser —(SELECT auth_token)—> Supabase
Browser <—(token plaintext)— Supabase
Browser —(?auth=TOKEN)—> CCTV Server

Token CCTV terlihat di:

🔍 Browser DevTools
(Network tab)

📄 JavaScript memory
(bisa di-extract)

📄 `<img src>` DOM
(visible in HTML)

💡 Penjelasan Sederhana

Kamu kasih kunci rumah orang ke kurir, suruh kurir buka pintu, lalu kurir tahu kuncinya selamanya. Harusnya **kamu sendiri** yang buka pintu, kurir cuma terima barang.

✅ Rekomendasi

Gunakan **backend proxy** — browser TIDAK PERNAH melihat token:

✓ ARSITEKTUR AMAN:

Browser —(minta stream camera X)→ Edge Function
Edge Function —(ambil token dari DB)→ Supabase
Edge Function —(inject token, fetch)→ CCTV Server
Edge Function ←(video stream)—— CCTV Server
Browser ←(forward stream)—— Edge Function

→ Browser tidak pernah lihat token!

KRITIS**R-03 — SSRF via User-Supplied Stream URL**

Masalah: User bisa isi `stream_url` dengan URL apapun, termasuk:

- `http://localhost:5432` — akses database internal
- `http://169.254.169.254` — AWS/GCP metadata (credentials)
- `http://internal-service:8080` — layanan internal

Jika backend proxy langsung fetch URL yang user berikan, penyerang bisa “menyuruh” server kamu mengakses resource internal.

💡 Penjelasan Sederhana

Bayangkan kamu punya sopir yang nurut sekali. Kamu kasih alamat, dia langsung kesana. Kalau ada orang jahat kasih alamat “gudang senjata”, sopir kamu tetap kesana. Berbahaya!

✓ Rekomendasi

- **Validasi URL:** hanya izinkan `https://` (block `http://`, `file://`, `ftp://`)
- **Blocklist:** tolak IP private ranges (10.x, 172.16.x, 192.168.x, 127.x, 169.254.x)
- **Allowlist lebih aman:** hanya izinkan domain yang didaftarkan admin
- **DNS resolution check:** resolve domain dulu, pastikan IP bukan internal

 **SEDANG****R-04 — Form Token Tanpa Masking**

Input token harus `type="password" + autocomplete="off"`. Tanpa ini, token terlihat saat diketik dan tersimpan di browser autofill.

 Rekomendasi**JSX — CamerasPage.jsx**

```
<input
  type="password"
  autoComplete="off"
  placeholder="Masukkan auth token..."
/>
```

 **SEDANG****R-05 — Mixed Content & CORS**

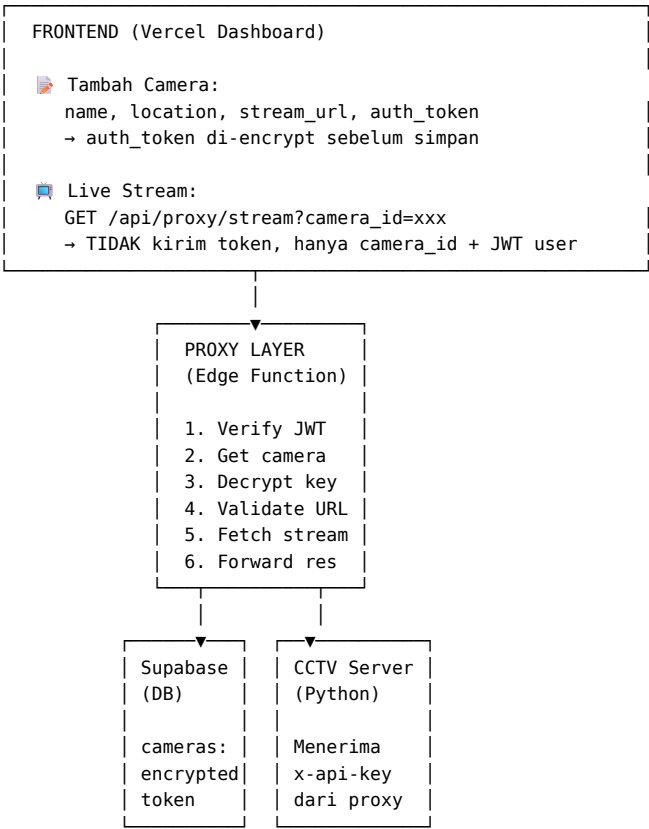
Dashboard HTTPS + CCTV server HTTP = browser block. CCTV server tanpa CORS header = fetch gagal.

 Rekomendasi

- Pastikan CCTV server selalu via HTTPS (Cloudflare Tunnel)
- Tambahkan CORS header di CCTV server, atau gunakan proxy (menghilangkan masalah CORS)

Arsitektur Rekomendasi

Desain Aman untuk Per-Camera Token



Checklist Implementasi

#	Checklist Item	Prioritas	Effort
1	Token di-encrypt at rest di database (BUKAN plaintext)	 KRITIS	4 jam
2	Token TIDAK pernah dikirim ke browser/frontend	 KRITIS	2 jam
3	Gunakan proxy (Edge Function) untuk inject token	 KRITIS	1 hari
4	Validasi stream_url (HTTPS only, block internal IP)	 KRITIS	2 jam
5	Input token di form pakai type="password"	 SEDANG	15 mnt
6	Rate limit proxy endpoint	 SEDANG	1 jam
7	Audit log: siapa akses stream kapan	 RENDAH	2 jam
8	Fitur "regenerate token" di UI	 RENDAH	1 jam

Rencana Aksi Gabungan (Bagian 1 + 2)

● Fase 1 — Perbaiki Middleware Saat Ini (Minggu 1)

No	Aksi	Effort
1	Tambah auth header di axios instance (api.js)	1 jam
2	Tambah auth di Socket.IO client + server	2 jam
3	Tambah VITE_SERVER_AUTH_KEY di .env dashboard	5 mnt
4	Update stream URL agar include auth param	30 mnt
5	Gunakan hmac.compare_digest di server	15 mnt

● Fase 2 — Implementasi Per-Camera Token (Minggu 2–4)

No	Aksi	Effort
6	Tambah kolom stream_url + auth_token_encrypted ke tabel cameras	1 jam
7	Buat Edge Function / Supabase Function sebagai stream proxy	1 hari
8	Update CamerasPage: form tambah stream_url + token (masked)	2 jam
9	Update LiveStreamingPage: konek via proxy, bukan langsung	2 jam
10	Validasi stream_url (HTTPS, block internal IP, SSRF protection)	2 jam

● Fase 3 — Hardening (Bulan 2+)

No	Aksi	Effort
11	Rate limiting pada proxy endpoint	1 jam
12	Content-Security-Policy header di Vercel	30 mnt
13	Audit log + alerting untuk akses stream	2 jam
14	Token rotation / regenerate UI	1 jam
15	Per-user JWT (untuk multi-operator)	2 hari



Kesimpulan

Rencana per-camera token arahnya sudah benar — ini evolusi yang tepat dari single static key.

Tapi ada **3 aturan emas** yang harus dipatuhi:

1. **Enkripsi token** di database — jangan simpan plaintext
2. **Proxy semua stream** lewat backend — browser tidak boleh pegang token
3. **Validasi URL** yang user masukkan — cegah SSRF

Urutan eksekusi:

- Fase 1 dulu (fix middleware saat ini) → 4 jam
- Fase 2 (per-camera token) → 2 hari
- Fase 3 (hardening) → ongoing

Jangan loncat ke Fase 2 sebelum Fase 1 selesai. Foundation harus kuat dulu.

Dokumen ini dihasilkan sebagai bagian dari audit keamanan internal.
Bukan merupakan nasihat hukum. Untuk compliance formal, konsultasikan dengan auditor bersertifikasi.